✨

Take-home Test

Overview

You are building the scheduling agent for a property maintenance company.

Each work order represents a real maintenance issue that requires a visit from a handyman. Your system must coordinate with the tenant and the handyman until it reaches one of two outcomes:

- `scheduled`
- `requires_review`

Work orders are provided through a read-only CSV file.

Communication with tenants and the handyman happens through stdin and stdout.

The system is delivered as a Docker container.

What your agent must do

For every work order:

1. Start the coordination process.
2. Communicate with the tenant and handyman as needed.
3. Reach one of these final outcomes:
    - `scheduled`
    - `requires_review`

A work order may only be `scheduled` when the tenant and handyman have both explicitly accepted the same visit date and time.

> ℹ️ When scheduled:

- Send a confirmation to both parties.
- Report the agreed datetime in the terminal result.

When coordination cannot be completed reliably, classify the work order as `requires_review` and provide a short reason.
Every work order must produce exactly one terminal result.
Once a terminal result has been emitted, that work order is complete.

Scheduling rules

- There is one handyman for all work orders.
- Each visit takes **2 hours**.
- Visits cannot overlap.
- When work orders compete for the same availability, higher urgency takes priority.
- Urgency levels are:
  - `Critical`
  - `High`
  - `Medium`
  - `Low`
- Always communicate with the handyman in Spanish.
- Communicate with the tenant in the language of the work order, Spanish or English.
- Use the `Europe/Madrid` timezone.
- Dates and times must be unambiguous.
- Multiple work orders may be active at the same time.
- Responses may arrive in any order.

A message whose body is exactly: `<NO RESPONSE>` represents a timeout (i.e. one of the parties did not respond), handle that the best way you consider. There might

be multiple responses of this kind. Do not use real-time waiting or wall-clock timeouts.

Startup behavior

When the container starts, the system must initiate coordination for **all work orders in the CSV**.

To simulate email communication, it must emit the initial outgoing message or messages for every work order without waiting for a response to a previous one.

For example:

```
WOID:WO-001|ROLE:tenant
Would tomorrow at 10:00 work for the handyman to visit?

WOID:WO-002|ROLE:tenant
When would you be available for the handyman to visit?

WOID:WO-003|ROLE:tenant
Would Wednesday afternoon work for you?
```

After startup, the application must remain ready to receive messages for any unfinished work order.

> ℹ️ Incoming messages may refer to any unfinished work order and may arrive in any order.
> The evaluator must be able to write at all times in the CLI (it is acceptable to wait for the output to print before being able to write again).

Deliverable

Provide a Git repository with your solution and a Dockerfile at its root.

Build with:

```
docker build -t agent .
```

Run with:

```
docker run --env-file .env -i \
  -v "$(pwd)/work_orders.csv:/app/work_orders.csv:ro" \
  -v "$(pwd)/traces:/app/traces" \
  agent
```

Docker is the only prerequisite on the evaluator's machine.

Evaluation

We evaluate the submission by running it.

During evaluation, we will provide a `work_orders.csv` file with **5 to 8 work orders** of mixed urgency.

Our evaluation system will:

1. Start the container.
2. Read outgoing messages from the terminal.
3. Send tenant and handyman responses through stdin.
4. Continue until every work order reaches a terminal result.

Scenarios may include:

- Clear availability
- Ambiguous responses
- Contradictory responses
- Availability changes
- Scheduling conflicts
- Missing responses
- Multiple simultaneous conversations

What we evaluate

- **Outcomes** — correct final result for each work order.
- **Communication** — clear, concise, appropriate language, unambiguous dates and times.
- **Scheduling correctness** — explicit agreement and no overlapping visits.
- **Robustness** — sensible handling of incomplete, unclear, contradictory, or missing responses.
- **Human intervention** — appropriate use of `requires_review`.
- **Engineering** — code structure, state handling, and error handling.
- **Traces** — enough information to reconstruct the coordination process.

> ✅ We care about how the agent coordinates, makes decisions, handles ambiguity, manages multiple conversations, and decides when human review is needed.
> The CSV and CLI interfaces are intentionally simple.

Work Order Input

The work-order file is available at: `/app/work_orders.csv`. It is read-only.

The schema is:

```
work_order_id,work_order,address,description,reported_at,urgency
```

Example:

```
work_order_id,work_order,address,description,reported_at,urgency
WO-001,Gas leak - 24 Oak Street,24 Oak Street,Tenant reports a strong smell of gas in the
kitchen,2026-10-07T09:15:00+02:00,Critical
WO-002,Water leak - 18 Calle Mayor,18 Calle Mayor,El inquilino informa de una fuga de agu
a debajo del fregadero,2026-10-07T10:40:00+02:00,High
```

```
WO-003,Bedroom door - 7 Pine Avenue,7 Pine Avenue,The bedroom door does not close properl
y,2026-10-07T12:05:00+02:00,Low
```

Your solution must work with different work orders following the same schema.

Communication Interface

Outgoing messages

Messages sent by the agent must use:

```
WOID:<work_order_id>|ROLE:<role>
<message body>
```

`<role>` is `tenant|handyman`

For outgoing messages, `ROLE` identifies the recipient.

Example:

```
WOID:WO-002|ROLE:handyman
¿Puedes realizar la visita mañana, 8 de octubre, a las 16:00?
```

Each message must end with a blank line.

Incoming messages

Incoming messages arrive through stdin:

```
<work_order_id>|<role> <message body>
```

For incoming messages, `<role>` identifies the sender.

Example:

```
WO-002|tenant Tomorrow after 17:00 works for me.
```

Responses may arrive in any order.

For example:

```
WO-003|tenant Wednesday afternoon works.
```

may be followed by:

```
WO-001|handyman Mañana a las 10:00 puedo.
```

and then:

```
WO-002|tenant <NO RESPONSE>
```

Terminal Results

A completed work order must emit one of two results.

Scheduled

```
WOID:<work_order_id>|RESULT:scheduled
SCHEDULED_FOR:<datetime>
```

Example:

```
WOID:WO-002|RESULT:scheduled
SCHEDULED_FOR:2026-10-08T18:00:00+02:00
```

Before emitting this result, confirmation messages must be sent to both the tenant and the handyman.

Requires Review

```
WOID:<work_order_id>|RESULT:requires_review
REASON:<reason>
```

Example:

```
WOID:WO-003|RESULT:requires_review
REASON:Unable to establish a mutually acceptable visit time.
```

⚠️ After a terminal result is emitted, no further messages or result changes should be produced for that work order.

Traces

The container must write structured traces to files under:

```
/app/traces
```

Use JSONL format, one JSON object per line.

Each trace event must contain at least:

- timestamp
- work order ID
- event type
- detail

The traces must cover:

- Incoming messages
- Outgoing messages
- Relevant state changes
- Relevant agent decisions

- Terminal results

Commit the traces from at least one complete run as an example.

> ⚠️ The terminal is reserved exclusively for the communication interface. Do not write traces, debug output, application logs, prompts, model responses, or other unrelated information to stdout or stderr during normal execution.
> All traces and runtime logs must be written to files under `/app/traces`.

Practical Details

- **Time:** 5 calendar days, always including a weekend.
- **LLM access:** We provide an Anthropic account separately.
- **Stack:** Any language, framework, or library.
- Document relevant assumptions and trade-offs in your README.
- For blocking issues, contact daniel.olea@nuvaliving.com.

Environment

Use `.env` for credentials required by your application.

Example:

```
ANTHROPIC_API_KEY=
```

Do not commit `.env`.